$\mathrm{tree}(4)$ の構成的下界

ε_0 での打切りと単一矢印の引数

$$H_{\Psi}(2 \uparrow^{4} 4) < \mathrm{tree}(4)$$

2026 年 10 月 18 日

定義・構成・任意値の証明・有限証明書を含む独立稿

概要

非空・無標識・無順序根付き木の、根の像を固定しない最小共通祖先保存埋め込みを用いる。第 i 項の頂点数を $i+4$ 以下とし、$H_{\Psi}(2\uparrow^{4}4)$ より長い不良列を構成する。添字 Ψ は第 1 節の式で固定し、その末尾は $\varepsilon_0\cdot 19$ である。入口 65 項、重み $(1,2,1)$ の準備、三・四座標の下降、順序数の下降に沿う二重カウンターを連結する。自然数引数は最初から $2\uparrow^{4}4$ とし、別の巨大入力の評価を経由しない。

192 個の低い区間の符号は、目印を結ぶ部分の自己同型と累積自然和によって一括して定義・証明する。有限部分の全データと停止する判定手続きを巻末に収録する。有限命題は全組の判定、無限のパラメータを含む命題は本文の一般証明で扱う。文献、過去の補論、別ファイル、計算結果ファイルを証明の前提としない。

目次

1 主張と記法 **1**
1.1 木と目標 . . . 1
1.2 順序数の表示と基本列 . . . 1

2 埋め込みを扱う三つの補題 **2**
2.1 穴、枠、代入の定義 . . . 2
2.2 比較の基本補題 . . . 2

3 epsilon 数より下の基本符号 **3**
3.1 表示の重み . . . 4

4 主部 L を符号化する有限型 **4**
4.1 穴の充填と順序反映 . . . 5

5 ν^2 の二区間と e の一区間 **5**

6 192 族を一度に扱う構成 **6**
6.1 有限表の意味 . . . 6
6.2 目印を結ぶ部分と定数枝の相殺 . . . 6
6.3 統一した充填規則 . . . 7

7 全域で共通に使うサイズ評価 **8**

8 有限入口と全後続族の分離 **9**
8.1 最初の準備列 . . . 9

9 三・四座標の下降で目標引数へ届く **10**
9.1 二つの族とその比較 . . . 10
9.2 重み付き下降の時間 . . . 10
9.3 一つの矢印の入力を直接使う . . . 11

10 Hardy 計算を木の列で実行する **11**

11 最大長の定義が有限値を持つこと **12**

A 有限命題の完全な証明書 **12**
A.1 式とコードの対応 . . . 12
A.2 再実行できる有限証明書 . . . 13

1　主張と記法

1.1　木と目標

$\mathbb{N}=\{0,1,2,\dots\}$ とする。p を一頂点の木、$[t_1,\dots,t_k]$ を新しい根に子木 $t_1,\dots,t_k$ を付けた木とする。子の順序は区別しないが、同じ形の子もその個数だけ別の子として数える。木は根付き同型を除いて扱う。$U(t)=[t]$、$P_n=U^{n-1}p$ $(n\ge 1)$、$M(t)=[p,p,t]$ と書く。ここで $U^0t=t$、$U^{k+1}t=U(U^kt)$ である。根から頂点 v への唯一の道上の頂点を v の祖先といい、v 自身も含める。二頂点 u,v の最小共通祖先 $u\wedge v$ は、共通祖先のうち根から最も遠いものをいう。$s\preccurlyeq t$ とは、頂点上の単射 $f:V(s)\to V(t)$ で $f(u\wedge v)=f(u)\wedge f(v)$ を全 u,v について満たすものが存在することである。根の像は任意であり、辺は長さ 1 以上の道に写ってよい。頂点数を $|t|$ とする。列 $T_1,\dots,T_\ell$ が合法な不良列であるとは

$$|T_i|\le i+4,\qquad i<j\Longrightarrow T_i\not\preccurlyeq T_j$$

をいう。その最大長を $\mathrm{tree}(4)$ と書く。最大値の存在は第 11 節で証明する。下界そのものの証明は、指定長を超える有限列の構成である。

以下で「s が族 $\mathcal{B}$ を避ける」は、全 $t\in\mathcal{B}$ について $s\not\preccurlyeq t$ の意味で使う。族どうしでも、前に記した族のどの木から後に記した族のどの木への埋め込みもない、という向きである。一方、「h 回避木 t」という用語は $h\not\preccurlyeq t$ を満たす t に限定する。単独の木どうしの回避は、向きを取り違えないよう以下では式で記す。単項・二分岐木とは、全頂点の子数が高々 2 の木であり、以下では純粋木とも呼ぶ。鎖とは子数が高々 1 の非空木、すなわち P_n である。木の族 $\mathcal{B}$ の単項・二分岐閉包は、$\mathcal{B}$ を含み、$t\mapsto[t]$ および $(s,t)\mapsto[s,t]$ で閉じる最小の木の族をいう。二分岐閉包とだけいう場合は後者の操作のみを許す。

順序数の和・積・冪は通常の演算とし、自然和だけを $\#$ で区別する。

$$\alpha=\varepsilon_{\omega^2\cdot 2+1},\quad x=\varepsilon_0,\quad e=\varepsilon_1,\quad \nu=e^{\omega},$$

$$P(z)=15z^4+75z^3+200z^2+240z+138,\qquad \mathcal{T}(f)(z)=\frac{f(z)^2+f(z^2)}{2},$$

$$Q=\mathcal{T}\big(\mathcal{T}(\mathcal{T}(P)+1)+1\big),\qquad L=\sum_{d=32,31,\dots,1}\alpha^d[z^d]Q(z),$$

$$\Gamma=L+\nu^2\cdot 2+e.$$

$[z^d]Q(z)$ は多項式 Q の z^d の係数を表す。和の順序は表示の通り降順である。以下の添字を固定する。

$$\begin{aligned}\Psi=\Gamma&+x^4\cdot 15+x^3(\omega\cdot 15+30)\\&+x^2(\omega^2\cdot 11+\omega\cdot 28+38)\\&+x(\omega^3\cdot 3+\omega^2\cdot 12+\omega\cdot 21+19).\end{aligned}\tag{1}$$

最後の項は $\omega^{\varepsilon_0}\cdot 19=\varepsilon_0\cdot 19$ であり、$\omega^2\Psi=\Psi$ である。

定理 1.1. 次節で定める基本列に対し

$$\boxed{H_\Psi\big(2\uparrow^4 4\big)<\mathrm{tree}(4)}.$$

1.2　順序数の表示と基本列

ε_η は $\omega^\xi=\xi$ の固定点を増加順に列挙したものである。必要な添字は $\eta\le\omega^2\cdot 2+1$ である。epsilon 数を原子として持つ遺伝的 Cantor 標準形を使い、$\omega^{\varepsilon_\eta}$ は ε_η へ正規化する。係数は同一主項の反復としても扱う。非零極限の最後の主項に対して

$$\begin{aligned}(\rho+\omega^\gamma c)[n]&=\rho+\omega^\gamma(c-1)+(\omega^\gamma)[n],\\(\omega^{\delta+1})[n]&=\omega^\delta(n+1),\\(\omega^\delta)[n]&=\omega^{\delta[n]}\quad(\delta\text{ が非零極限})\end{aligned}$$

とする。ただし正規化した主項が epsilon 原子なら、次の原子規則を優先する。

$$W_0(a) = a, \quad W_{j+1}(a) = \omega^{W_j(a)},$$
$$\varepsilon_0[n] = W_{n+1}(1), \quad \varepsilon_{\eta+1}[n] = W_{n+1}(\varepsilon_\eta + 1), \quad \varepsilon_\lambda[n] = \varepsilon_{\lambda[n]} \quad (\lambda \text{ が非零極限}).$$

原子添字の基本列は、ω の有限多項式に対する上の Cantor 規則である。したがって、たとえば正の前部 ρ を持つ $\rho + \varepsilon_0 c$ の末項にも $\varepsilon_0[n]$ を用いる。この規約を計算の全段で固定する。

$$H_0(n) = n, \quad H_{\beta+1}(n) = H_\beta(n+1), \quad H_\lambda(n) = H_{\lambda[n]}(n).$$

状態 (β, n) をこの式に従って更新し、$\beta = 0$ で停止する。各更新で順序数が真に減るため停止し、最終引数が Hardy 値である。以下で Hardy 関数の添字に関する単調性は使わない。

自然和は Cantor 項の指数をそろえて係数を足す演算である。有限個の自然和は結合的・可換で、各変数について狭義単調である。これは最初に異なる係数を比較すれば分かる。特に $u \# v \le u \# w$ なら $v \le w$。epsilon 数未満、および e^ω 未満は有限自然和で閉じる。同じ長さのタプルの辞書順は、最初に異なる座標で比較する順序をいう。自然数座標は通常の大小、順序数座標は順序数の大小で比較する。

2　埋め込みを扱う三つの補題

2.1　穴、枠、代入の定義

穴とは、後で一つの非空木に置き換えるための記号 X_i であり、木の頂点ではない。枠は、次の再帰で作る有限式である。各 X_i は枠であり、枠 $C_1, \dots, C_k$ から $[C_1, \dots, C_k]$ も枠となる（$k \ge 0$）。空の括弧 [] は固定頂点 p である。各括弧は一つの固定頂点を表す。子式の順序は無視するが、その多重度は保存する。一つの枠の中では穴の各出現に異なる識別子を付ける。識別子を省いて X と書いても、各出現は独立した代入位置である。穴がちょうど一つの枠を一穴枠という。穴がない枠は普通の木である。

木 $t_1, \dots, t_d$ の代入（または充填）を $C(t_1, \dots, t_d)$ と書く。$\mathbf{t} = (t_1, \dots, t_d)$ として、再帰式は

$$X_i(\mathbf{t}) = t_i, \qquad [C_1, \dots, C_k](\mathbf{t}) = [C_1(\mathbf{t}), \dots, C_k(\mathbf{t})]$$

である。穴の位置には代入木の根そのものを置き、接続用の余分な頂点は加えない。例えば $[X_1, [p, X_2]](s, t) = [s, [p, t]]$ である。固定頂点数を $\mathrm{fix}(C)$、穴数を $d(C)$ とすると

$$\mathrm{fix}(X_i) = 0, \quad d(X_i) = 1, \qquad \mathrm{fix}([C_1, \dots, C_k]) = 1 + \sum_j \mathrm{fix}(C_j), \quad d([C_1, \dots, C_k]) = \sum_j d(C_j),$$

従って $|C(t_1, \dots, t_d)| = \mathrm{fix}(C) + \sum_i |t_i|$ である。穴の深さは式の根からその出現までの括弧の段数で、X_1 だけなら 0 である。穴を一時的な葉として扱うと道・祖先・最小共通祖先も定義できる。これは枠を調べるための形式的な位置であり、代入後の頂点数には加えない。

穴の種類を指定するときは、その穴へ代入できる木の族を指定している。純粋穴には純粋木、鎖穴には鎖を入れる。任意の二つの枠を接合する際は穴の識別子を付け直し、独立性を保つ。穴番号を忘れた型の一致は、固定頂点・穴の種類を保つ根付き同型で定める。本文の有限型では、この同型で同じものを一度だけ数える。

2.2　比較の基本補題

補題 2.1 (再帰判定)**.** $s \preccurlyeq t$ は、s が t の一つの子木へ入るか、s の子を t の相異なる子へ割り当てて各組で埋め込めることと同値である。

証明. 根の像が対象の根より下なら第一の場合である。根同士が対応するとき、別の子の像は最小共通祖先保存により別の子枝に入る。逆に各枝の単射を根で接合すれば必要な単射を得る。　□

補題 2.2 (一穴枠の相殺)**.** 固定した一穴木枠 F に対し、任意の木 s,t について $F(s) \preccurlyeq F(t)$ と $s \preccurlyeq t$ は同値である。

証明. 深さ 0 では $F(s)=s$ なので明らかである。穴の深さに関する帰納法を用いる。穴を含む子枠を G、他の固定子木を B_i とする。$F(s)$ はどの B_i よりも大きいので、対象の固定子には潜れない。$G(t)$ へ潜る場合、$G(s) \preccurlyeq F(s) \preccurlyeq G(t)$ と帰納仮定から結論を得る。根同士が対応する場合、子割当ては置換である。穴を持つ子を通る置換の巡回をたどると、固定子を経由するか否かにかかわらず $G(s) \preccurlyeq G(t)$ を推移性から得る。再び帰納仮定を使う。逆向きは枠への代入である。 □

補題 2.3 (整列した対応)**.** 全順序集合の $a_1 \le \cdots \le a_d$ と $b_1 \le \cdots \le b_d$ に対し、置換 π が $a_i \le b_{\pi(i)}$ を満たせば $a_i \le b_i$ がすべて成立する。

証明. $a_i > b_i$ なら、源の末尾 $d-i+1$ 個を対象の末尾高々 $d-i$ 個へ写す必要が生じる。 □

3 epsilon 数より下の基本符号

$\lambda = \omega^2 \cdot 2$ とし、$D_{ab} = [P_a, P_b]$ と置く。

$$q_{\omega a+b} = D_{a+1,a+b+1}, \quad q_{\omega^2+\omega a+b} = U D_{a+1,a+b+1}, \quad q_\lambda = U^2 D_{11}.$$

$\xi < \alpha$ の符号 $\mathcal{C}(\xi)$ を

$$\mathcal{C}(0) = p, \qquad \mathcal{C}(\varepsilon_\eta) = M(q_\eta)$$

とし、非零・非原子的な $\xi = \omega^{\xi_1} + \cdots + \omega^{\xi_r}$ では $\xi_1 \ge \cdots \ge \xi_r$ とし、有限係数の個数だけ同じ指数を反復して書く。$t = \mathcal{C}(\xi_1)$ から始め、$j = 1, \dots, r$ の順に $t \leftarrow [\mathcal{C}(\xi_j), t]$ とする。得られた t を $\mathcal{C}(\xi)$ とする。各 $\xi_j < \xi$ なので、この再帰は定義を与える。以下で原子木という場合は、この規則で ε_η へ割り当てた固定木 $M(q_\eta)$ を指す。同じ二分岐再帰で、原子を持たない $\xi < x$ の符号を $\mathcal{C}_0$、$\xi < e$ で唯一の原子 x を $A = [p,p,p]$ へ写す符号を $\mathcal{C}_1$ とする。

$$h = M(U^2 D_{12}), \qquad g = M(U^2 D_{13}), \qquad A_2 = M(P_2).$$

$h \not\preccurlyeq \mathcal{C}(\xi)$ であり、$\mathcal{C}_0(\xi)$ は単項・二分岐木、$\mathcal{C}_1(\xi)$ は $\{p, A\}$ の二分岐閉包に属する。

補題 3.1. これら三つの符号は順序を反映する。すなわち $\mathcal{C}_i(\xi) \preccurlyeq \mathcal{C}_i(\eta) \Rightarrow \xi \le \eta$ であり、$\mathcal{C}$ でも同じである。ここで $i = 0, 1$ で、各写像について両引数はその定義域に属するものとする。

証明. q の唯一の分岐点の上の幹と、昇順の二本の腕を比較すると $q_\eta \preccurlyeq q_\theta \Rightarrow \eta \le \theta$ となる。幹長 $0, 1, 2$ はこの順序を逆行できず、幹長が等しければ $a \le a'$、$a + b \le a' + b'$ が必要である。最後の幹長 2 の型もこの結論を満たす。h の幹長 2・腕長 $(1, 2)$ はどの q にも入らない。

原子木の値を対応する epsilon 数、葉の値を 0 とし、二分岐では

$$v([s,t]) = \omega^{\min(v(s),v(t))} \# \max(v(s), v(t))$$

とする。この演算は各変数について非減少で、両変数より真に大きい。原子の内部の原子を持たない部分木は、単項を $v \mapsto v+1$ で評価すれば ε_0 未満の値を持つ。原子の埋め込みは先ほどの添字比較を満たす。原子を持つ外側の二分岐木が一つの原子の内部へ入ることはない。その三分岐点が対象原子の根へ写ると、その真の祖先を写す場所がないからである。補題 2.1 の帰納法により、符号木同士の埋め込みは v を減らさない。

$E(\xi) = v(\mathcal{C}(\xi))$ と置くと、非原子的な場合

$$E(\xi) = E(\xi_1) \# \omega^{E(\xi_1)} \# \cdots \# \omega^{E(\xi_r)}.$$

E は狭義単調である。最大指数が異なるとき、小さい側は $\omega^{E(\xi_1)+1}$ 未満、大きい側の先頭項はそれ以上である。最大指数が等しいとき、共通の最初の自然和項を除いて指数列を辞書式に比較する。原子との比較では、epsilon 数未満が有限自然和と ω 冪で閉じることを用いる。これで構造帰納法が閉じ、主張を得る。$\mathcal{C}_0, \mathcal{C}_1$ も同じ証明である。 □

3.1 表示の重み

$\eta=\omega^2a+\omega b+c$ に対して $I(\eta)=a+b+c+1$ とし

$$\mathrm{w}(0)=1,\quad \mathrm{w}(\varepsilon_\eta)=10+4I(\eta),\quad \mathrm{w}(\omega^{\xi_1}+\cdots+\omega^{\xi_r})=1+\sum_{j=1}^{r}(\mathrm{w}(\xi_j)+1)$$

とする。最後の式は非零・非原子的な標準形に適用する。

補題 3.2. 必要な順序数の全域で、後続前者は重みを増やさず、非零極限では

$$\mathrm{w}(\beta[n])\le(n+2)\,\mathrm{w}(\beta).$$

また $|\mathcal{C}(\xi)|,|\mathcal{C}_0(\xi)|,|\mathcal{C}_1(\xi)|\le 4^{\mathrm{w}(\xi)}$ である。

証明. 原子の前者添字は I を増やさない。極限添字の一段の近似では I の増加は高々 n である。$W_{n+1}(1)$ の重みは $2n+5$。後続 epsilon の近似では、前の原子の重みに高々 $2n+6$ が加わる。これらは原子の主張を満たす。通常の Cantor 尾では、後続指数なら最後の指数の前者を持つ項を $n+1$ 個作り、極限指数ならその指数へ帰納仮定を適用する。共通前部の重みは保存され、原子への正規化は重みを減らす。後続前者は末尾の 1 を消すだけである。

符号の原子のサイズは順に $2a+b+6, 2a+b+7, 8$ 以下で、対応する 4^{w} 以下。非原子のサイズは $|\mathcal{C}(\xi_1)|+\sum_j(|\mathcal{C}(\xi_j)|+1)$ である。重みの和の 4 冪がこの和を抑えるので、構造帰納法で結論を得る。 □

4 主部 L を符号化する有限型

第 2.1 節の枠を使い、穴の識別子を省いて X と書く。$\mathrm{Pair}(V)=\{[s,t]:s,t\in V\}$ は重複使用を許す無順序対の集合である。$s=t$ の場合も二つの別のコピーを用い、穴はコピー間で共有しない。$S_2=[p,p]$ とし、以下を定める。

$$\begin{aligned}
B&=\{[X,[X,U^ih]]:i=0,1\}\\
&\quad\cup\{[[X,U^ih],[X,U^jh]]:(i,j)=(0,0),(0,1),(1,1)\},\\
Z_*&=\{U[p,h],U[P_2,h],[h,h],[h,Uh],[Uh,Uh]\},\\
E_1&=\{[X,z],[z,[X,h]],[z,[X,Uh]]:z\in Z_*\},\\
V&=B\cup E_1\cup\mathrm{Pair}(Z_*)\cup\{U^2[S_2,h]\},\\
G&=\mathrm{Pair}(V)\cup\{U[P_k,U[S_2,h]]:k=1,2\},\\
R&=U[S_2,U[S_2,h]],\qquad K_0=UR,\qquad E_0=U[p,R],\\
\mathcal{H}&=\mathrm{Pair}(G)\cup\{K_0\},\quad \mathcal{F}=\mathrm{Pair}(\mathcal{H})\cup\{E_0\},\quad \mathcal{A}=\mathrm{Pair}(\mathcal{F}).
\end{aligned}$$

穴数 $d(t)$ と固定頂点数 $\mathrm{fix}(t)$ は第 2.1 節の定義による。ここでの記号比較 $s\preccurlyeq_* t$ は、次の順序で適用する再帰的な真偽判定である。対象 t が穴なら、源 s の固定子式の中に h そのものが現れないとき真とする。対象が穴でなく源 s が穴なら、対象に穴が一つ以上あるとき真とする。両者が括弧なら、源が対象のある子式と比較できるか、源の子式を対象の相異なる子式へ割り当てて各組を比較できるとき真とする。それ以外は偽である。固定子式とは穴を含まない子式であり、h が現れるとはその式が木 h と同型であることをいう。これは充填木の埋め込みそのものの定義ではなく、補題 4.3 で必要条件として使う判定である。

補題 4.1 (有限型の比較)**.** $|V|=36$、$|G|=668$。V 内の比較は穴数を保存し、異なる型なら固定費用を増やす。$\mathrm{Pair}(V)$ は V へ入らず、G の追加 *2* 型とは双方向に比較不能である。追加 *2* 型内の比較も固定費用を増やす向きだけで、その対は追加 *2* 型へ入らない。$K_0,R,E_0,[p,R]$ は G へ入らず、$E_0,[p,R]$ は K_0 へ入らない。G の穴数 $0,1,2,3,4$ の個数は $138,240,200,75,15$、固定費用は *87* 以下である。

証明. 巻末の `V,G0,extra,G` は上の集合を全て生成し、`sym` は定義通りの記号比較を行う。全称検査はそれぞれ 36^2 組、$666\cdot 36$ 組、$2\cdot 666\cdot 2$ 組など有限個の全組を含む。各再帰呼出しでは木の構造が小さくなり、判定は停止する。全データ、再帰、各命題に対応する `assert` を第 A 節に示す。 □

命題 4.2. $s,t\in\mathcal{A}$ に対し $s\preccurlyeq_* t$ なら $d(s)=d(t)$、異なる型なら $\mathrm{fix}(s)<\mathrm{fix}(t)$。穴数生成関数は Q であり、固定費用は 703 以下である。異なる二穴の最小共通祖先は固定の h を子孫に持つ。

証明. $\mathrm{Pair}(V)\not\preccurlyeq_* V$ から、対を重ねた同一階層での比較は根同士に限られる。子へ潜る場合も源の子を取り出すと、下の階層の禁止比較を生むからである。G の追加型は h を一つ、$\mathrm{Pair}(V)$ は二つ以上含む。別々の h の像は別々の三分岐点へ写り、穴へは入らない。この個数制約と補題 4.1 により $\mathrm{Pair}(G)\not\preccurlyeq_* G$。$K_0$ が $\mathrm{Pair}(G)$ へ入るなら K_0 または R が G へ入り矛盾する。逆向きは h の個数で排除できる。同様に $E_0,[p,R]$ の禁止比較を使うと、$\mathrm{Pair}(\mathcal{H})\not\preccurlyeq_* \mathcal{H}$、$\mathrm{Pair}(\mathcal{F})\not\preccurlyeq_* \mathcal{F}$ となる。したがって各階層の比較は子同士の比較へ帰着し、穴数保存と固定費用の主張が帰納的に上がる。

無順序対の生成関数は $(f(z)^2+f(z^2))/2$ である。異なる二要素は平方で二度、同じ要素は一度数えられるためである。三回の対と二つの追加型から Q を得る。費用は $87,175,351,703$ と増える。穴の最小共通祖先の主張は B,E_1 で直接確認でき、対を取る操作で保たれる。 □

4.1 穴の充填と順序反映

各穴へ $Z(\xi)=[P_{1024},U^{1024}\mathcal{C}(\xi)]$、$\xi<\alpha$ を入れる。これは $h\not\preccurlyeq Z(\xi)$ を満たし、二本の腕は固定部分全体より長い。

補題 4.3. 充填した型に埋め込みがあれば、元の型に記号比較がある。穴数が等しければ穴は全単射で対応し、対応するパラメータについて $\xi\le\eta$ となる。

証明. 源の Z は固定部分だけに入らず、対象の穴へ達する。対象の穴に入る源の部分へ h は埋め込めないので、記号比較の両穴規則が必要条件になる。二つの源穴が同じ対象穴を使うなら、その最小共通祖先の下の固定 h も対象穴へ入り矛盾する。穴数が等しいと、各源穴が使う対象穴は一つずつで全単射である。源 Z の根が対象の固定部分に残るなら、その二本の長い腕が別々の穴に達し、この全単射に反する。したがって対応する Z 同士の内部で比較する。$U^{1024}\mathcal{C}(\xi)$ は P_{1024} より長く、その腕へは入らない。対応する長い腕を相殺するか、対象の $\mathcal{C}(\eta)$ 内部へ潜る場合を取れば $\mathcal{C}(\xi)\preccurlyeq\mathcal{C}(\eta)$ が必要である。補題 3.1 を用いる。 □

穴数 $d=32,31,\dots,1$ の順に、同じ穴数の型を固定費用の昇順に並べる。同費用の型は一つの固定順に並べ、各型に長さ α^d の区間を割り当てる。具体的には、穴を文字 X、頂点を子式の括弧列で表し、各頂点で子の文字列を辞書順に整列した標準文字列の辞書順を使えばよい。各型について一つの整列済み代表を固定し、その式を左から読む順に穴へ番号を付ける。証明はこの固定順の選択に依存しない。区間内を $\alpha^{d-1}c_1+\cdots+c_d$ と書き、固定順に番号を付けた第 i 穴へ

$$Z(c_1\,\#\cdots\#\,c_i)$$

を入れる。得られる符号を $F(\beta)$、$\beta<L$ とする。型順の逆比較は命題 4.2 で排除される。同じ型では補題 4.3 と 2.3 から累積自然和の成分比較が必要であり、最初に異なる生の桁で共通自然和を取り除けば生の桁列の辞書順を反映する。よって $F(s)\preccurlyeq F(t)\Rightarrow s\le t$ が $[0,L)$ 全体で成立する。

5 ν^2 の二区間と e の一区間

次の固定枠を使う。

$$D_0=U[P_2,U[S_2,U[S_2,h]]],\quad J(t)=[t,D_0],\quad B_k(t)=[p,[A_2,[t,U^kA_2]]],\quad \mathcal{W}(t)=J(B_1(t)).$$

$|D_0|=23$、$|J(t)|=24+|t|$、$|B_k(t)|=14+k+|t|$、$|\mathcal{W}(t)|=39+|t|$。巻末の `assert not match(D0,ff)` は、任意の h 回避木を穴に入れた $\mathcal{F}$ のどの木にも D_0 が入らないことを判定している。したがって任意の $J(t)$ は $[0,L)$ の符号を避ける。根を合わせても潜らせても、D_0 が一つの $\mathcal{F}$ の木へ入る必要が生じるためである。

$c<\nu$ を先頭非零の e 進表示 $c=e^{d-1}c_1+\cdots+c_d$ で書く。$c=0$ は空列とする。末端 A_2 から逆順に $t\leftarrow[\mathcal{C}_1(c_i),t]$ とした木を $Z_e(c)$ とする。これは A_2 をちょうど一つ含む。$\mathcal{C}_1$ の三分岐は A であり、A_2 はそこへ入らない。従って $Z_e(c)\preccurlyeq Z_e(c')$ なら幹長 $d\le d'$。$d<d'$ なら $c<c'$、$d=d'$ なら各側木が対応し $c_i\le c'_i$ となる。よって Z_e は順序反映である。

$c,d<\nu$ について $Y(c,d)=[Z_e(c),Z_e(c\#d)]$ とする。二つの A_2 を持つので根を片腕へ潜らせられず、腕の置換と補題 2.3 により $\nu c+d$ の順序を反映する。

$$F(L+\nu^2k+\nu c+d)=J(U^kY(c,d))\qquad(k=0,1),$$
$$F(L+\nu^2\cdot 2+a)=J(B_0(\mathcal{C}_1(a)))\qquad(a<e).$$

同じ区間内の比較は相殺補題と今の議論による。第 2 の ν^2 区間から第 1 へは、二つの A_2 の最小共通祖先の上の一段の幹により入らない。B_0 ではその最小共通祖先の上に二分岐がある。これを Y または UY へ写すとその祖先の像を置く場所がないため、e 区間から前二区間へも入らない。さらに B_1 の二つの A_2 の深さは 2 と 4、B_0 では 2 と 3 なので、任意の t,u が p,A の単項・二分岐閉包に属するとき $B_1(t)\not\preccurlyeq B_0(u)$ である。同じ祖先の議論は $B_1(t)$ から Y,UY も排除する。これで $\mathcal{W}(t)$ を用いる後続すべてと前部の分離が得られる。

6　192 族を一度に扱う構成

6.1　有限表の意味

巻末の `D` は、行番号 0 から始める木の有限表である。整数だけの行 $(i_1,\dots,i_k)$ は先行行を子にした節点を表し、空行 `()` は葉を表す。`('hole',i)` は任意の単項・二分岐木を入れる純粋穴、`('chain',i)` は任意の非空鎖を入れる鎖穴である。`PAT_IDS` が 192 個の内部枠を以下の順に指定する。

番号（0 始まり）	純粋穴 d	鎖穴 k	個数	一区間の長さ
0–14	4	0	15	x^4
15–29	3	1	15	$x^3\omega$
30–59	3	0	30	x^3
60–70	2	2	11	$x^2\omega^2$
71–98	2	1	28	$x^2\omega$
99–136	2	0	38	x^2
137–139	1	3	3	$x\omega^3$
140–151	1	2	12	$x\omega^2$
152–172	1	1	21	$x\omega$
173–191	1	0	19	x

この順に Γ の後へ連結すると終端は正確に Ψ となる。表の枠は三分岐 A と単項・二分岐だけを持ち、各穴番号は各種類内で一度だけ現れる。

6.2　目印を結ぶ部分と定数枝の相殺

枠の固定子式 $A=[p,p,p]$ の各出現を目印という。枠にある全ての目印の根と全体の根を結ぶ道の合併を、道の骨格とする。その頂点から出る子式のうち、目印を含まないものを縁の枝という。目印自身の内部の三枚の葉は縁の枝に含めない。比較用の骨格は、目印を色 A の一頂点へ、各縁の枝を色 v

の一頂点へ縮約した根付き木である。残りの固定頂点には色 n を付ける。色 v の頂点には、その枝の出現を区別する番号を付ける。骨格の自己同型は根・隣接関係・三種類の色を保存する頂点の全単射である。枝の番号やその内容は保存する条件に含めず、番号上に誘導される置換を記録する。巻末の `skeleton` と `maps` は、この骨格とその自己同型を再帰的に全列挙する。

補題 6.1. *192* 枠すべてについて、縁の枝は純粋穴、鎖穴、固定純粋木のいずれかである。骨格の自己同型は全部で *363* 通りある。各自己同型の縁の枝の置換から固定枝を巡回内で除くと、純粋穴は純粋穴へ、鎖穴は鎖穴へ写る。

証明. `check_skeleton` が全 192 枠でこの命題を判定する。`maps` は根の子のすべての置換を取り、各子の同型を再帰的に接合する。葉型の一致と子数の一致を要求するので、過不足なく全自己同型を列挙する。各巡回で次の変数枝まで進んで種類が同じことを検査し、合計も 363 と照合する。全定義とデータは巻末にある。 □

補題 6.2 (任意値での骨格比較)**.** 一つの枠の二つの充填木が埋め込みで比較できるなら、純粋穴の値同士と鎖穴の値同士に、それぞれ埋め込みを満たす全単射が存在する。

証明. 源の A の根は対象の A の根にしか写れない。個数が等しいので全単射である。各 A の根の深さは減らせず、全深さの和は源と対象で同じである。従って全ての深さが保たれ、全体の根も根へ写る。A へ向かう各辺は一辺へ写らなければならないため、骨格に自己同型を誘導する。その縁の枝の比較は、対応する純粋部分木同士の埋め込みである。

変数枝から始め、置換先が固定枝ならその固定枝を源として次の比較へ進む。源と対象で固定枝は同一なので、推移性により途中の固定枝を消去できる。各巡回から固定枝を消す操作は変数枝上の置換を与える。補題 6.1 によりその置換は穴の種類を保存する。ゆえに両種類について必要な全単射を得る。この議論は枝の大きさや穴の値を制限しない。 □

6.3 統一した充填規則

純粋穴 d 個、鎖穴 k 個の枠の一区間内の順序数を

$$x^{d}(\omega^{k-1}n_1 + \cdots + n_k) + x^{d-1}c_1 + \cdots + c_d$$

と書く。$k = 0$ なら最初の括弧はなく、$n_i \in \mathbb{N}$、$c_i < x$ である。各種類の穴をその番号の昇順に並べ、第 i 純粋穴と第 j 鎖穴へそれぞれ

$$\mathcal{C}_0(c_1 \,\#\, \cdots \,\#\, c_i), \qquad P_{1+n_1+\cdots+n_j}$$

を代入し、外側に $\mathcal{W}$ を付ける。これをその区間の F と定める。

命題 6.3. この定義は $[\Gamma, \Psi)$ 全体の順序を反映する。

証明. 同じ枠では補題 6.2 と 2.3 から累積自然数和と累積自然和の成分比較が必要である。最初に異なる自然数座標、またはそれらが同じなら最初に異なる順序数桁を比較し、共通累積和を取り除けば $(n_1, \ldots, n_k, c_1, \ldots, c_d)$ の辞書順を反映する。これは表示した順序数の比較そのものである。

異なる枠について、全穴に葉を入れた種木は同じ枠の全充填木へ入る。巻末では第 i 枠の種木から第 $j < i$ 枠の任意穴充填への存在判定を全組で否定している。したがって後の枠から前の枠への埋め込みはない。外枠 $\mathcal{W}$ は補題 2.2 で相殺できる。前部との分離は第 5 節により、これで $[0, \Psi)$ 全体の順序反映が成立する。 □

7　全域で共通に使うサイズ評価

ここでは区間ごとに異なる重みを導入せず、全標準形の重みを使う。

$$S(\beta) = \mathrm{w}(\beta) + 128, \qquad A_\beta = 2^{1000(S(\beta)+1000)}.$$

補題 7.1. $\beta < \Psi$、$a \geq 0$、$j \geq 1$ に対し

$$|F(\omega^2\beta + \omega a + j)| \leq A_\beta + 4a + 2j.$$

符号の引数は Ψ 未満である。また *Hardy* 更新では $S(\beta') \leq (n+2)S(\beta)$、$n' \leq n+1$ である。

証明. 最後の主張は補題 3.2 から従う。任意の $\beta < \Psi$ について $\omega^2\beta + \omega a + j < \omega^2(\beta+1) \leq \omega^2\Psi = \Psi$。左からの ω^2 倍は各 Cantor 指数 ξ を $2+\xi$ へ変える。無限指数は変わらず、有限指数の重み増加は 4 以下である。従ってカウンターを除いた重みは高々 $5\,\mathrm{w}(\beta)+2$ でよい。

符号に渡す各パラメータの費用を確認する。以下で $\gamma = \omega^2\beta$ と置き、$\mathrm{w}(\gamma) \leq 5\,\mathrm{w}(\beta)$ を使う。α 進の有限 32 桁を取り出す操作は、Cantor 指数から共通の α 倍を消すだけである。非零桁を別々の標準形へ分ける際の外側の 1 と零桁の追加費用を合わせても高々 64。したがって生の全桁の重みは $\mathrm{w}(\gamma) + 64 \leq 6S$ 以下である。e 進表示の桁数は指数に含まれる有限係数に 1 を加えた値以下で、全重みで抑えられる。二つの ν 進係数を取り出し、それぞれを e 進へ分ける場合も、非零桁を別の標準形へ分ける費用と零桁の費用は元の重みの二倍に定数 6 を足した値以下である。従って両係数の生の e 進桁の重みの和は $3\,\mathrm{w}(\gamma) + 6 \leq 16S$ 以下となる。$x^d\omega^k$ の区間では自然数座標は Cantor 係数であり、その和は全重み以下、純粋桁の重みの和も共通前部を除いた重みに桁ごとの外側の 1 と零桁の費用を加えた値以下である。純粋桁は高々 4 個なので、後者は $\mathrm{w}(\gamma) + 8 \leq 6S$ 以下。区間内の係数から固定した区間番号を引く操作はこれらを増やさない。

自然和は指数列を併合するだけで、原子を一つの Cantor 項へ開く費用は一原子につき 2 以下。従って主部の一つの累積桁の重みは $6S + 64 \leq 7S$ 以下。ν 部の e 進桁の自然和は桁ごとに行え、その重みの和は元の両桁列の重みの和の三倍以下だから $48S$ 以下。低い 192 族では一つの累積桁の重みは $6S + 8 \leq 7S$ 以下であり、三本以下の累積鎖の総可変長は $15S$ 以下。基本符号の個数は主部で 32 以下、ν 部で両桁列の長さの和以下、低い族で 4 以下である。以上をまとめると、各基本符号の重みは $100S$ 以下、基本符号の個数と鎖の総可変長の和も $100S$ 以下である。固定費用は L 部で $703 + 32 \cdot 2049 = 66271$、他はこの値より小さい。最後の生の順序数桁への $\omega a + j$ の追加は、累積和のうち最後の純粋穴一つだけを変える。ν 部でも右腕の最後の e 進桁一つだけを変える。この穴のカウンターを加える前の値を κ とする。非原子の折り畳みでは、末尾の ω 一項は $|\mathcal{C}_i(1)| + 1 = 4$ 頂点、1 一項は $|\mathcal{C}_i(0)| + 1 = 2$ 頂点を加える（$\mathcal{C}$ でも同じ）。先頭指数が変わるのは $\kappa < \omega^2$ の場合だけで、その初期の芯は 7 頂点以下。ただし κ が epsilon 原子の場合、最初の項を折り畳むと $[\mathcal{C}(\kappa), \mathcal{C}(\kappa)]$ となる。この追加費用は $|\mathcal{C}(\kappa)| + 1$ であり、定数 7 で抑えてはならない。前の 4^{100S} 評価でこの一コピーも数え、零桁からの新しい腕と初期の芯も過大に数えると、カウンターの $4a + 2j$ 以外の全費用は

$$(200S + 140010)4^{100S} + 16 < 2^{1000(S+1000)} = A_\beta \qquad (S \geq 129)$$

以下である。従って主張の線形費用が得られる。 □

補題 7.2 (初期重み)**.** $\mathrm{w}(\Psi) < 2^{128}$。$q \geq 1024$ なら、$\beta_0 = \Psi[q]$ に対して

$$X_0 = S(\beta_0) + q + 10000 < 2^{130}(q+2), \qquad 2^q \geq \left(2^{130}(q+2)\right)^2.$$

証明. $\mathrm{w}(\alpha) = 26$。α^d の指数 αd の重みは高々 $27d+1$ なので、L の重みは $867Q(1)+1$ 以下である。残りの表示は係数が高々 38、次数が高々 4 で、定義を展開すると重みは 10^6 未満。$Q(1) > 1000$ なので全体は $10000Q(1)$ 未満。巻末の整数多項式計算が $10000Q(1) < 2^{128}$ を確認する。基本列の重み評価から $X_0 < 2^{130}(q+2)$。最後の不等式は $q = 1024$ で整数計算により確認し、q を 1 増やすと左辺は二倍、右辺の増加率は二倍未満なので帰納する。 □

8　有限入口と全後続族の分離

巻末の PRE_IDS は、同じ有限表 D で 65 個の固定木を順に指定する。最後の木の定義を明記するため

$$V_0 = [p, P_3], \quad Q_{30}(t) = \mathcal{W}([[t, U^2 A], U[V_0, UA]]), \quad Q_{33}(t) = Q_{30}([t, A])$$

と置く。第 65 項は $Q_{33}([p,[p,p]])$ である。巻末の EXTRA_IDS は順に、Q_{33} の内部一穴枠、その全外枠、次節の四鎖族の全パターン、三座標族の全パターンを指定する。

補題 8.1. *65 項は合法な不良列で、全項が $F([0,\Psi))$ を避ける。最初の 64 項は Q_{33} の全純粋充填を避ける。*

証明. サイズ 65 個と先行 2080 対を補題 2.1 で全検査する。穴付きパターンへの存在判定 match は、純粋穴では純粋性、鎖穴では鎖であることを判定し、節点では全子割当てと潜り込みを調べる。独立穴なので各成功部分で選んだ代入を一緒にでき、この再帰は必要十分である。全 65 項から 192 族へ、また最初の 64 項から Q_{33} への判定は全て偽となる。

L 部については最初の 12 項を別に扱う。$\mathcal{C}$ の全原子と固定 h は、巻末の atoms で表す三種類の二鎖原子および h に含まれる。その単項・二分岐閉包への存在判定が wide であり、最初の 12 項を全て排除する。第 13 項以降は、少なくとも一つの穴を持つ $\mathcal{A}$ への存在判定 positive(aa) が全て偽となる。穴では任意の h 回避木を許しているため、実際のガード付き充填も全て排除される。

中間二区間には、p, A の単項・二分岐閉包を T1、A_2 の上にその側木を任意個付ける文法を Z として使う。固定の源 s について、Z への存在は、$s \preccurlyeq A_2$、s が T1 へ入る、または子を T1 と Z へ割り当てられる、のいずれかである。右の Z へ単に潜る場合は同じ判定へ戻るだけなので新しい成功条件にならない。それ以外の再帰は源の真の子に移るため停止する。この文法で $J([Z,Z]), J(U[Z,Z]), J(B_0(T1))$ を表した nu0,nu1,ep への判定も全 65 項で偽となる。これで全ての区間を尽くした。 □

8.1　最初の準備列

$$C(u,v,w) = U^u[P_{v+1}, P_{v+w+1}], \qquad |Q_{33}(C(u,v,w))| = 67 + u + 2v + w.$$

一つの分岐点の幹と二本の昇順の腕の比較により、C の埋め込みは $u \le u', v \le v', v+w \le v'+w'$ と同値であり、辞書順を反映する。第 65 項の穴の木は分岐点二個、C は一個なので、相殺補題により前者は後者へ入らない。$Q_{33}(p)$ の内部種木は全 192 族の任意充填を避けることを巻末で全判定する。前部との分離は $\mathcal{W}$ の共通の議論による。

重み列 $(w_1, \dots, w_d)$、初期予算 b に対する有限列を次のように定義する。空の重み列は空タプル一個。$b = w_1 a + r$、$0 \le r < w_1$ とし、第一座標を $a, a-1, \dots, 0$ とする。第 j ブロック開始までの出力数を T として、残りの重みに予算 $r + w_1 j + T$ を渡す。次元に関する帰納法から、この列は有限、辞書順に真下降し、第 s 項の重み和は $b+s$ 以下である。

第 66 項から $(1,2,1)$、予算 3 の列を $Q_{33}(C)$ へ入れる。第 s 項（$s=0$ 始まり）のサイズは $70+s = (66+s)+4$ 以下。重み $(2,1)$、予算 $2a+r$ の列長は

$$\ell_{21}(2a+r) = (r+3)(2^{a+1}-1) - 2(a+1), \quad r = 0, 1.$$

実際、内部累計 T_j は $T_{j+1} = 2T_j + r + 2j + 1$ を満たし、$T_j = (r+3)(2^j - 1) - 2j$ を代入すれば確認できる。$(1,2,1)$ の四ブロックの累計は $T_0 = 0$、$T_{j+1} = T_j + \ell_{21}(j + T_j)$ から

$$1, \quad 6, \quad 89, \quad M = 3 \cdot 2^{47} - 8.$$

従ってこの段の終了時刻は $t_0 = 65 + M$。これ以外の大きな準備は必要ない。

9　三・四座標の下降で目標引数へ届く

9.1　二つの族とその比較

$B(t)=[t,UA]$、$D=B(S_2)$ とし

$$G_4(u,v,w,z)=\mathcal{W}([[B(P_{u+1}),B(P_{u+v+1})],[B(P_{u+v+w+1}),B(P_{u+v+w+z+1})]]),$$
$$H(u,v,w)=\mathcal{W}([[B(P_{u+1}),B(P_{u+v+1})],[B(P_{u+v+w+1}),D]]).$$

$$|G_4|=70+4u+3v+2w+z,\qquad |H|=72+3u+2v+w.$$

補題 9.1. G_4 の比較は $(u,u+v,u+v+w,u+v+w+z)$ の成分比較と同値であり、H の比較は $(u,u+v,u+v+w)$ の成分比較と同値である。全 H は全 G_4 を避け、両族は $F([0,\Psi))$ を避ける。入口と最初の準備列の全項は両族を避ける。

証明. $B(P_{n+1})$ の唯一の三分岐 A の対応から、その比較は $n\le m$ と同値である。二対の根は A の個数により潜らせられず、子の置換のみを考えればよい。昇順の四数では、各対の交換も二対全体の交換も、比較が可能なら最小同士、次同士の成分比較を与える。これは補題 2.3 でも分かる。D はどの $B(P_{n+1})$ にも入らない。S_2 を鎖へ入れられないためである。従って H 同士では D の位置が固定され、残り三鎖を同様に比較する。また全 H から全 G_4 への比較も排除される。

$H(0,0,0)$ と $G_4(0,0,0,0)$ の種木から 192 族への任意穴判定は全て偽。前部との分離は $\mathcal{W}$ に共通である。入口の最初の 64 項と $Q_{33}(S_2)$ から、任意鎖長を許す H,G_4 のパターンへの判定も全て偽。第 65 項と全準備項は $Q_{33}(S_2)$ を含むので、この有限判定がすべての値を排除する。 □

9.2　重み付き下降の時間

生成済み項数を t、固定費用を c（H で 72、G_4 で 70）とする。固定済みの上位座標の重み和を W、残る座標数を d として、

$$z=t+6-c-W,\qquad q=\left\lfloor\frac{z-1}{d}\right\rfloor$$

と置く。次の座標を $q,q-1,\dots,0$ とし、その都度残りの座標を現在の時刻で同じように初期化する。残り 0 座標なら一つの木を出し t を 1 増やす。選択時に重み和は $t+5-c$ 以下なので全項が合法であり、辞書順真下降と補題 9.1 から不良列となる。終了時のずらした予算を $L_d(z)$ と書くと

$$L_1(z)=2z,\qquad L_d(dq+r)=(u\mapsto L_{d-1}(u+d))^{\langle q\rangle}(L_{d-1}(r)),\quad 1\le r\le d.$$

下位ブロック終了後、上位座標を 1 減らすと予算が d 増えるためである。この再帰は有限列を定義し、その列長は $L_d(z)-z$ である。

$F_j=H_{\omega^j}$、$h_j(n)=F_j(n-1)+1$（$n\ge1$）とする。有限回の ω^j の和では最後の項から Hardy 更新が進むことから

$$F_1(n)=2n+1,\quad F_2(n)=2^{n+1}(n+1)-1,\quad F_{j+1}(n)=F_j^{\langle n+1\rangle}(n),\quad h_1(n)=2n,\quad h_{j+1}(n)=h_j^{\langle n\rangle}(n).$$

これらの関数は再帰式から狭義単調である。

補題 9.2. $1\le d\le4$、$n\ge1$ に対し $L_d(64n)\ge64h_d(n)$。

証明. まず次元への帰納法で L_d は非減少、$L_d(z)\ge2z$。同じ商 q の中の単調性は明らかで、剰余 d から次の剰余 1 への境界では $R(R(1)+d)\ge R(d)$（$R=L_{d-1}$）を使う。下界は $R(r)\ge2r$、$R(u+d)\ge u+2d$ から $2r+2dq=2z$ を得る。

$d=1$ の主張は等号。$d\ge2$ では R の呼出しが少なくとも $16n$ 回ある。最初の $8n$ 回の倍増で $2^{8n}\ge64n$ に達し、残る呼出しのうち n 回に帰納仮定を用いる。各回の $+d$ を捨てても $64h_{d-1}^{\langle n\rangle}(n)=64h_d(n)$ に達する。$2^{8n}\ge64n$ は $n=1$ で成り立ち、左辺の増加率 256 が右辺の増加率以下にはならない。 □

9.3　一つの矢印の入力を直接使う

底 2 の矢印を $A_1(n)=2^n$、$A_k(0)=1$（$k\ge 2$）、$A_{k+1}(n+1)=A_k(A_{k+1}(n))$ と定める。$A_k(n)=2\uparrow^k n$ である。$T=A_2$、$P_*=A_3$、$Q_*=A_4$ と略記すると

$$T(4)=65536,\quad P_*(3)=65536,\quad P_*(4)=T(65536)=Q_*(3).$$

各 A_k は狭義単調であり、$A_k(n)>n$ を同時に帰納すれば従う。$F_2(v)\ge 2^{v+1}$ から反復により $F_3(n)\ge T(n+1)$、さらに $F_4(n)\ge P_*(n+1)$（$n\ge 1$）となる。

$$q=2\uparrow^4 4,\qquad a=65536,\qquad n=F_3(a)$$

と置くと

$$n\ge T(65537)>T(65536)=Q_*(3),\qquad F_4(n)\ge P_*(n+1)>P_*(Q_*(3))=q.$$

最初の準備後は $z_0=t_0-66\ge 64(a+2)$。H を全三座標で下降させ、続いて G_4 を全四座標で下降させる。両終了時刻は補題 9.2 から

$$\begin{aligned} t_H &= L_3(z_0)+66,\\ z_G=t_H-64 &= L_3(z_0)+2\ge 64(F_3(a+1)+1)+2\ge 64(n+2),\\ t_1=L_4(z_G)+64 &\ge 64(F_4(n+1)+1)+64>F_4(n+1)>F_3(F_4(n))>F_3(q). \end{aligned}$$

最後から二つ目の不等式は、$F_4(n+1)$ が F_3 を $n+2$ 回、より大きい入力から反復することによる。この時点までの全列は合法かつ不良で、全項が後続の $F([0,\Psi))$ を避ける。

10　Hardy 計算を木の列で実行する

最初の状態を $(\beta_0,q)=(\Psi[q],q)$ とする。Ψ は極限なので $H_{\beta_0}(q)=H_\Psi(q)$。ここでは $\beta_0<\Psi$ であり、準備に使った H,G_4 を再使用しない。$q>1024$ かつ $F_3(q)>2^{2^q}$ なので、補題 7.2 と前節から

$$t_1>2^{X_0^2},\qquad X_0=S(\beta_0)+q+10000.$$

$F_3(q)>2^{2^q}$ は F_2 の最初の二反復だけでも得られ、残りの反復で真に増える。

補題 10.1 (一段の出力)**.** 状態 (β,r)、生成済み項数 t について

$$X=S(\beta)+r+10000,\qquad t\ge 2^{X^2}$$

とする。$a=\lfloor t/16\rfloor,\dots,0$ の各ブロック開始時刻を s とし

$$j=\left\lfloor\frac{s-A_\beta-4a}{2}\right\rfloor,\dots,1$$

の順に $F(\omega^2\beta+\omega a+j)$ を出力する。全項は合法で、終了時刻 $t_{\mathrm{new}}>2^{t/100}$。*Hardy* 更新後の状態でも同じ不変量が保たれる。

証明. $X\ge 10129$ なので $1000(S+1000)+2<X^2$、従って $A_\beta\le t/4$。$4a\le t/4$、$s\ge t$ よりブロック長は $s/4-1\ge s/5$。補題 7.1 から各木のサイズはブロック開始時刻 s 以下であり、合法である。一ブロックで時刻は 6/5 倍以上になる。ブロック数は $t/16$ より多い。$6^{100}>2^{16}5^{100}$ より $t_{\mathrm{new}}>2^{t/100}$。

更新後は $S'\le(r+2)S$、$r'\le r+1$ なので $X'\le X^4$。$X\ge 10129$ では $2^{X^2}/100>X^8$（$X=10129$ で成立し増加率を比較すればよい）なので $t_{\mathrm{new}}>2^{(X')^2}$ となる。 □

定理*1.1*の証明. 非零状態ごとに補題 10.1 の有限列を出力し、その後 Hardy 更新を一回行う。一状態内では (a, j) が辞書順に真下降する。状態が $\beta' < \beta$ へ下がると、任意の $\rho < \omega^2$ について

$$\omega^2\beta' + \rho < \omega^2(\beta' + 1) \le \omega^2\beta.$$

したがって全符号引数は真下降し、F の順序反映により後続列は不良である。準備との分離は第 8–9 節により、全連結列が合法な不良列となる。Hardy 計算は停止する。停止時の自然数引数を r_* とすると $r_* = H_{\Psi[q]}(q) = H_\Psi(q)$。不変量は停止状態にも保たれているため、その時点の列長 t は

$$t \ge 2^{(S(0)+r_*+10000)^2} > r_*.$$

よって $\mathrm{tree}(4) \ge t > H_\Psi(2 \uparrow^4 4)$ である。 □

11　最大長の定義が有限値を持つこと

ここは最大長という表記を閉じるための補足であり、上の構成や数値評価には使わない。準順序が整列準順序であるとは、無限列に先行する項から後続項への比較が必ずあることをいう。そのような無限列は無限非減少部分列を持つ。実際、任意の無限添字集合内に無限個の比較可能な後続を持つ項がある。なければ、既に選んだ有限個の項の有限な比較先を避け続けて不良列を作れてしまう。この性質を選んだ項の後続集合へ繰り返し適用する。

整列準順序上の有限語も、文字の比較を許す部分列埋め込みで整列準順序となる。そうでなければ、無限不良列へ延長可能という条件で各位置の語長を最小にした列を選ぶ。非空語を $w_i = v_i a_i$ と書き、末尾文字の無限非減少部分列 a_{i_j} を取る。$w_0, \dots, w_{i_0-1}, v_{i_0}, v_{i_1}, \dots$ も不良列である。前部から後部への比較は元の比較を生み、後部同士の比較には末尾文字を足せば元の比較を生む。しかし v_{i_0} は短く、最小選択に反する。

木に無限不良列があるとして、同様に各位置の頂点数を最小にした列 T_i を選ぶ。これらの根の直接の子木全体を S とする。S に不良列があれば、出所の添字を真に増加する k_j にできる。初めの有限個の木からは有限個の子しか出ず、不良列で同じ木を反復できないからである。$T_0, \dots, T_{k_0-1}, s_0, s_1, \dots$ は不良となり、$|s_0| < |T_{k_0}|$ が最小性に反する。従って S は整列準順序。各 T_i の子を任意の固定順に並べた語に前段の結果を使い、枝の比較を根で接合すると $T_i \preccurlyeq T_j$ を得て矛盾する。

合法な有限不良列を接頭関係で並べると有限分岐の木になる。長さに上限がなければ、任意に長く延長できる子を各段で選べる。そのような子が一つもなければ、有限個の子の延長長の最大値が親の上限となるからである。これは無限不良列を与えて矛盾する。従って合法な長さの集合は有界で、有限な最大値を持つ。

A　有限命題の完全な証明書

A.1　式とコードの対応

通常の木は、その子の符号のタプルで表す。葉は空タプルであり、`tree` は子を一つの固定順へ整列して無順序の同型を同じ符号にする。`size` は固定された木の頂点数、`emb` は補題 2.1 の判定である。枠を扱う別の符号では、`N(f1,...,fk)` が固定頂点とその子枠を表す。以下のタグを持つパターンは、その位置に許される木の族を指定する。

タグ	許される木または操作
`hole`	任意の純粋木
`chain`	任意の非空鎖
`free`	$h \not\preccurlyeq t$ を満たす任意の木 t
`T1`	$\{p, A\}$ の単項・二分岐閉包
`Z`	A_2 の上に `T1` の側木を有限個付けた木
`or`	後続の子パターンが表す族の和集合
`node`	各子パターンから独立に選んだ木を新しい根へ接続

`Z` の族を厳密に書けば、A_2 を含み、$z \mapsto [t, z]$ （t は `T1` の族の任意の木）で閉じる最小の族である。この表のパターン f が表す族を $\mathcal{L}(f)$ とすると、`match(s,f)` は $\exists t \in \mathcal{L}(f)\ (s \preccurlyeq t)$ の真偽を返す。`T1(s)`、`Z(s)`、`wide(s)` も所属そのものではなく、指定した族のどれかへ s が埋め込めるかという判定である。`sym` だけは第 4 節の記号比較であり、`match` とは区別する。

`D` の各行は先行行だけを参照する有向非巡回の記述である。同じ行を何度参照しても各参照位置に式を展開した別のコピーを置き、頂点や穴を共有するグラフにはしない。`seed` は純粋穴と鎖穴に葉 p を代入する操作である。`positive(aa)` は、少なくとも一つの自由穴を持つ型だけを残した主部のパターンである。巻末コード内の `None` は、主部の小さい有限型を計算するときの穴記号 X にだけ用いる。

A.2　再実行できる有限証明書

以下の Python 3 コードは標準ライブラリだけを使用し、外部入力を読まない。有限表は探索規則ではなく、今回使用する木と枠そのものの定義である。各 `assert` は本文に記した有限命題の全称判定に対応する。骨格の一般補題、重み評価、下降の時間、Hardy 計算は本文で証明し、標本実験で代用していない。コード全体を一つのファイルに写し、最適化オプションを付けずに実行すると末尾の成功文を出す。TeX から取り出す場合は、この付録の `Verbatim` 環境の内容をそのまま保存すればよい。

```
from functools import lru_cache
from itertools import permutations,product

def tree(*cs):return tuple(sorted(cs,key=repr))
p=()
def U(t,k=1):
 for _ in range(k):t=(t,)
 return t
def P(n):return U(p,n-1)
def M(t):return tree(p,p,t)
A=M(p);A2=M(P(2));S2=tree(p,p)
h=M(U(tree(p,P(2)),2));g=M(U(tree(p,P(3)),2))
D0=U(tree(P(2),U(tree(S2,U(tree(S2,h))))))
def J(t):return tree(t,D0)
def BK(t,k):return tree(p,tree(A2,tree(t,U(A2,k))))
def whole(t):return J(BK(t,1))

@lru_cache(None)
def size(t):return 1+sum(map(size,t))
@lru_cache(None)
def emb(s,t):
 if size(s)>size(t):return False
 return any(emb(s,z) for z in t) or any(
```

```
  all(emb(x,t[j]) for x,j in zip(s,pi))
  for pi in permutations(range(len(t)),len(s)))
@lru_cache(None)
def pure(s):return len(s)<=2 and all(map(pure,s))
@lru_cache(None)
def chain(s):return len(s)<=1 and all(map(chain,s))
@lru_cache(None)
def T1(s):return emb(s,A) or len(s)<=2 and all(map(T1,s))
@lru_cache(None)
def Z(s):
 return emb(s,A2) or T1(s) or len(s)<=2 and any(
  all((T1,Z)[j](x) for x,j in zip(s,pi))
  for pi in permutations(range(2),len(s)))
def N(*cs):return ('node',)+cs
def fixed(t):return N(*(fixed(z) for z in t))
@lru_cache(None)
def match(s,f):
 tag=f[0]
 if tag=='hole':return pure(s)
 if tag=='chain':return chain(s)
 if tag=='free':return not emb(h,s)
 if tag=='T1':return T1(s)
 if tag=='Z':return Z(s)
 if tag=='or':return any(match(s,z) for z in f[1:])
 cs=f[1:]
 return any(match(s,z) for z in cs) or any(
  all(match(x,cs[j]) for x,j in zip(s,pi))
  for pi in permutations(range(len(cs)),len(s)))
def union(*cs):return ('or',)+cs
def wrap(f):
 return N(fixed(D0),N(fixed(p),N(fixed(A2),N(f,N(fixed(A2))))))
def seed(f):
 return p if f[0] in ('hole','chain') else tree(*(seed(z) for z in f[1:]))

# Small high-level templates; None is a hole.
X=None
@lru_cache(None)
def cost(t):return 0 if t is X else 1+sum(map(cost,t))
@lru_cache(None)
def deg(t):return 1 if t is X else sum(map(deg,t))
@lru_cache(None)
def marked(t):return t is not X and (t==h or any(map(marked,t)))
@lru_cache(None)
def sym(s,t):
 if t is X:return not marked(s)
 if s is X:return deg(t)>0
 return any(sym(s,z) for z in t) or any(
  all(sym(x,t[j]) for x,j in zip(s,pi))
  for pi in permutations(range(len(t)),len(s)))
def pair(seq):
 seq=sorted(seq,key=repr)
```

```
 return {tree(x,y) for i,x in enumerate(seq) for y in seq[i:]}
B={tree(X,tree(X,U(h,e))) for e in (0,1)}
B|={tree(tree(X,U(h,e)),tree(X,U(h,f)))
    for e,f in ((0,0),(0,1),(1,1))}
tags={U(tree(p,h)),U(tree(P(2),h)),tree(h,h),
      tree(h,U(h)),tree(U(h),U(h))}
E1={tree(X,z) for z in tags}
E1|={tree(z,tree(X,U(h,e))) for z in tags for e in (0,1)}
V=B|E1|pair(tags)|{U(tree(S2,h),2)}
G0=pair(V)
extra={U(tree(P(k),U(tree(S2,h)))) for k in (1,2)}
G=G0|extra
R=U(tree(S2,U(tree(S2,h))))
K=U(R);E=U(tree(p,R))
assert (len(V),len(G),max(map(cost,G)))==(36,668,87)
assert all(not sym(s,t) or (deg(s)==deg(t) and
 (s==t or cost(s)<cost(t))) for s in V for t in V)
assert all(not sym(s,t) for s in G0 for t in V)
assert all(not sym(s,t) and not sym(t,s) for s in extra for t in G0)
assert all(not sym(s,t) or s==t or cost(s)<cost(t)
 for s in extra for t in extra)
assert all(not sym(s,t) for s in pair(extra) for t in extra)
assert all(not sym(s,t) for s in (K,R,E,tree(p,R)) for t in G)
assert not sym(E,K) and not sym(tree(p,R),K)
assert [sum(deg(t)==d for t in G) for d in range(5)]==[138,240,200,75,15]
def pat(t):return ('free',) if t is X else N(*(pat(z) for z in t))
gg=union(*(pat(t) for t in G))
hh=union(N(gg,gg),fixed(K));ff=union(N(hh,hh),fixed(E));aa=N(ff,ff)
@lru_cache(None)
def positive(f):
 if f[0]=='free':return f
 if f[0]=='or':
  terms=tuple(q for z in f[1:] if (q:=positive(z)) is not None)
  return union(*terms) if terms else None
 cs=f[1:];terms=[]
 for i,z in enumerate(cs):
  q=positive(z)
  if q is not None:terms.append(N(*(cs[:i]+(q,)+cs[i+1:])))
 return union(*terms) if terms else None
assert not match(D0,ff)

# All automorphisms of the subtree joining the A markers to the root.
FA=fixed(A)
def hasA(f):return f==FA or f[0]=='node' and any(hasA(z) for z in f[1:])
def skeleton(f,fringe):
 if not hasA(f):
  i=len(fringe);fringe.append(f);return ('v',i)
 if f==FA:return ('A',)
 return ('n',)+tuple(skeleton(z,fringe) for z in f[1:])
def maps(s,t):
 if s[0]=='v' and t[0]=='v':return [((s[1],t[1]),)]
```

```
 if s[0]!=t[0] or len(s)!=len(t):return []
 if s[0]=='A':return [()]
 out=[]
 for pi in permutations(range(1,len(t))):
  for ms in product(*(maps(z,t[j]) for z,j in zip(s[1:],pi))):
   out.append(sum(ms,()))
 return out
def no_variables(f):return f[0]=='node' and all(no_variables(z) for z in f[1:])
def check_skeleton(f):
 def grammar(z):
  return z[0] in ('hole','chain') or z==FA or len(z)<=3 and all(grammar(t) for t
   ↪  in z[1:])
 assert grammar(f) and hasA(f)
 fringe=[];s=skeleton(f,fringe)
 kinds=[z[0] if z[0] in ('hole','chain') else 'fixed' for z in fringe]
 assert all(k!='fixed' or no_variables(z) for k,z in zip(kinds,fringe))
 for kind in ('hole','chain'):
  ids=[z[1] for z in fringe if z[0]==kind]
  assert len(ids)==len(set(ids))
 mm=maps(s,s)
 for m in mm:
  d=dict(m)
  for i,k in enumerate(kinds):
   if k=='fixed':continue
   j=d[i]
   while kinds[j]=='fixed':j=d[j]
   assert kinds[j]==k
 return (kinds.count('hole'),kinds.count('chain'),len(mm))

# Finite data. Each row references earlier rows.
D=(
 ('hole',0),('hole',1),(),(2,2,2),(1,3),(0,4),('hole',2), # 0..6
 ('hole',3),(7,3),(6,8),(5,9),(8,),(6,11),(5,12),(6,3), # 7..14
 (14,8),(5,15),(14,11),(5,17),(14,),(19,11),(5,20),(4,), # 15..22
 (0,22),(23,12),(23,15),(23,17),(23,20),(0,3),(28,4), # 23..29
 (29,15),(29,17),(29,20),(28,22),(33,17),(33,20),(28,), # 30..36
 (36,22),(37,20),('chain',0),(3,),(39,40),(6,41),(5,42), # 37..43
 (23,42),(42,29),(42,33),(42,37),(41,14),(5,48),(23,48), # 44..50
 (48,29),(48,33),(48,37),(41,19),(5,54),(23,54),(54,29), # 51..57
 (54,33),(54,37),(2,2),(60,40),(6,61),(5,62),(23,62), # 58..64
 (62,29),(62,33),(62,37),(14,61),(5,68),(23,68),(29,68), # 65..71
 (33,68),(68,37),(19,61),(5,74),(23,74),(29,74),(33,74), # 72..78
 (37,74),(2,),(2,80),(81,40),(6,82),(5,83),(23,83),(83,29), # 79..86
 (83,33),(83,37),(14,82),(5,89),(23,89),(29,89),(33,89), # 87..93
 (89,37),(19,82),(5,95),(23,95),(29,95),(33,95),(37,95), # 94..100
 (0,41),('chain',1),(102,40),(1,103),(101,104),(103,4), # 101..106
 (101,106),(103,22),(101,108),(41,28),(110,106),(110,108), # 107..112
 (41,36),(113,108),(41,103),(5,115),(23,115),(115,29), # 113..118
 (115,33),(115,37),(1,61),(101,121),(121,110),(121,113), # 119..124
 (4,61),(101,125),(110,125),(113,125),(22,61),(101,129), # 125..130
 (110,129),(113,129),(1,82),(101,133),(133,110),(133,113), # 131..136
```

```
(4,82),(101,137),(110,137),(113,137),(22,82),(101,141), # 137..142
(110,141),(113,141),(41,61),(5,145),(23,145),(145,29), # 143..148
(145,33),(145,37),(41,82),(5,151),(23,151),(151,29), # 149..154
(151,33),(151,37),(0,61),(157,121),(157,125),(157,129), # 155..160
(157,133),(157,137),(157,141),(28,61),(164,125),(164,129), # 161..166
(133,164),(164,137),(164,141),(36,61),(170,129),(133,170), # 167..172
(137,170),(170,141),(0,82),(175,133),(175,137),(175,141), # 173..178
(28,82),(179,137),(179,141),(36,82),(182,141),(61,61), # 179..184
(5,184),(23,184),(29,184),(33,184),(37,184),(61,82), # 185..190
(5,190),(23,190),(29,190),(33,190),(37,190),(82,82), # 191..196
(5,196),(23,196),(29,196),(33,196),(37,196),(80,),(2,202), # 197..203
(203,40),(204,),(5,205),(29,205),(0,103),('chain',2), # 204..209
(209,40),('chain',3),(211,40),(210,212),(208,213),(103,28), # 210..215
(215,213),(210,11),(115,217),(210,61),(208,219),(210,82), # 216..221
(208,221),(103,210),(157,223),(175,223),(215,219), # 222..226
(215,221),(223,164),(223,179),(115,74),(115,95),(103,19), # 227..232
(145,232),(151,232),(208,184),(208,190),(208,196),(103,61), # 233..238
(157,238),(103,82),(157,240),(175,238),(175,240),(215,184), # 239..244
(215,190),(215,196),(238,164),(240,164),(238,179), # 245..249
(240,179),(103,36),(251,184),(251,190),(251,196),(238,170), # 250..255
(238,182),(240,170),(240,182),(157,184),(157,190), # 256..260
(157,196),(175,184),(175,190),(175,196),(164,184), # 261..265
(164,190),(164,196),(179,184),(179,190),(179,196),(61,36), # 266..271
(271,184),(271,190),(271,196),(82,36),(275,184),(275,190), # 272..277
(275,196),(40,),(0,279),(280,205),(2,2,2,2),(2,80,80), # 278..283
(2,2,3),(2,60),(2,2,285),(60,),(287,),(288,),(2,2,289), # 284..290
(80,80),(291,),(292,),(2,2,293),(202,),(2,295),(296,), # 291..297
(297,),(2,2,298),(203,),(300,),(2,2,301),(2,302),(302,), # 298..304
(304,),(305,),(81,),(307,),(2,2,308),(309,),(310,),(311,), # 305..312
(312,),(313,),(314,),(315,),(316,),(2,309),(2,318),(319,), # 313..320
(320,),(321,),(322,),(323,),(309,202),(325,),(326,),(327,), # 321..328
(328,),(329,),(330,),(3,309),(332,),(333,),(334,),(335,), # 329..336
(336,),(337,),(60,309),(339,),(2,340),(2,341),(342,), # 337..343
(343,),(344,),(202,340),(346,),(347,),(348,),(349,), # 344..350
(3,340),(351,),(352,),(353,),(354,),(60,340),(356,), # 351..357
(2,357),(2,358),(359,),(202,357),(361,),(362,),(3,357), # 358..364
(364,),(365,),(60,357),(367,),(80,368),(2,368),(368,), # 365..371
(2,2,60),(80,357),(373,),(372,374),(2,2,295),(376,374), # 372..377
(2,2,202),(378,),(379,),(374,380),(80,378),(382,374), # 378..383
(2,378),(384,374),(379,374),(378,374),(2,2,80),(388,388), # 384..389
(389,),(390,),(374,391),(80,389),(393,374),(388,), # 390..395
(395,395),(2,396),(397,374),(395,),(399,),(388,400), # 396..401
(2,401),(402,374),(2,388),(2,404),(388,405),(2,406), # 402..407
(407,374),(60,388),(409,),(388,410),(2,411),(412,374), # 408..413
(388,295),(414,),(388,415),(2,416),(417,374),(388,202), # 414..419
(419,),(388,420),(2,421),(422,374),(80,388),(424,), # 420..425
(388,425),(2,426),(427,374),(404,),(388,429),(2,430), # 426..431
(431,374),(358,),(433,433),(279,),(435,),(436,),(395,437), # 432..438
(388,438),(2,439),(440,374),(2,3),(2,442),(443,),(444,), # 439..445
(395,445),(388,446),(2,447),(448,374),(3,3),(450,),(451,), # 446..452
(395,452),(388,453),(2,454),(455,374),(287,40),(457,), # 453..458
```

```
 (458,),(395,459),(388,460),(2,461),(462,374),(291,40), # 459..464
 (464,),(465,),(395,466),(388,467),(2,468),(469,374), # 465..470
 (296,40),(471,),(472,),(395,473),(388,474),(2,475), # 471..476
 (476,374),(2,204),(2,478),(479,395),(388,480),(2,481), # 477..482
 (482,374),(60,204),(484,),(395,485),(388,486),(2,487), # 483..488
 (488,374),(204,295),(490,),(395,491),(388,492),(2,493), # 489..494
 (494,374),(202,204),(496,),(395,497),(388,498),(2,499), # 495..500
 (500,374),(80,204),(502,),(395,503),(388,504),(2,505), # 501..506
 (506,374),(478,),(395,508),(388,509),(2,510),(511,374), # 507..512
 (2,40),(513,),(514,205),(395,515),(388,516),(2,517), # 513..518
 (518,374),(442,),(2,520),(521,205),(395,522),(388,523), # 519..524
 (2,524),(525,374),(60,3),(527,),(528,),(205,529),(395,530), # 525..531
 (388,531),(2,532),(533,374),(3,295),(535,),(536,), # 532..537
 (205,537),(395,538),(388,539),(2,540),(541,374),(357,), # 538..543
 (543,543),(433,544),(3,202),(546,),(547,),(205,548), # 544..549
 (395,549),(388,550),(2,551),(552,374),(80,3),(554,),(555,), # 550..556
 (205,556),(395,557),(388,558),(2,559),(560,374),(520,), # 557..562
 (562,205),(395,563),(388,564),(2,565),(566,374),(279,279), # 563..568
 (205,568),(395,569),(388,570),(2,571),(572,374),(3,285), # 569..574
 (574,279),(575,205),(395,576),(388,577),(2,578),(579,374), # 575..580
 (28,279),(581,205),(395,582),(388,583),(2,584),(585,374), # 581..586
 (115,213),(395,587),(388,588),(2,589),(590,374),(115,219), # 587..592
 (395,592),(388,593),(2,594),(595,374), # 593..596
)
PAT_IDS=(
 10,13,16,18,21,24,25,26,27,30,31,32,34,35,38,43,
 44,45,46,47,49,50,51,52,53,55,56,57,58,59,63,64,
 65,66,67,69,70,71,72,73,75,76,77,78,79,84,85,86,
 87,88,90,91,92,93,94,96,97,98,99,100,105,107,109,111,
 112,114,116,117,118,119,120,122,123,124,126,127,128,130,131,132,
 134,135,136,138,139,140,142,143,144,146,147,148,149,150,152,153,
 154,155,156,158,159,160,161,162,163,165,166,167,168,169,171,172,
 173,174,176,177,178,180,181,183,185,186,187,188,189,191,192,193,
 194,195,197,198,199,200,201,206,207,214,216,218,220,222,224,225,
 226,227,228,229,230,231,233,234,235,236,237,239,241,242,243,244,
 245,246,247,248,249,250,252,253,254,255,256,257,258,259,260,261,
 262,263,264,265,266,267,268,269,270,272,273,274,276,277,278,281,
)
PRE_IDS=(
 282,283,284,286,290,294,299,303,306,305,304,302,317,324,331,338,
 345,350,355,360,363,366,369,370,371,375,377,381,383,385,386,387,
 392,394,398,403,408,413,418,423,428,432,434,441,449,456,463,470,
 477,483,489,495,501,507,512,519,526,534,542,545,553,561,567,573,
 580,
)
EXTRA_IDS=(
 582,586,591,596,
)

decoded=[]
for row in D:
```

```
 decoded.append(row if row and isinstance(row[0],str)
                else N(*(decoded[i] for i in row)))
patterns=tuple(decoded[i] for i in PAT_IDS)
prefix=tuple(seed(decoded[i]) for i in PRE_IDS)
prep,prepat,gpat,hpat=(decoded[i] for i in EXTRA_IDS)
v0=fixed(tree(p,P(3)));fa=fixed(A);bp=lambda z:N(z,N(fa))
q33=N(N(N(('hole',0),fa),N(N(fa))),N(N(v0,N(fa))))
assert seed(prep)==seed(q33)
assert not no_variables(prep)
assert seed(prepat)==whole(seed(prep))
rawg=N(N(bp(('chain',0)),bp(('chain',1))),
       N(bp(('chain',2)),bp(('chain',3))))
rawh=N(N(bp(('chain',0)),bp(('chain',1))),
       N(bp(('chain',2)),bp(fixed(S2))))
def canon(f):
 return f if f[0]!='node' else ('node',)+tuple(sorted((canon(z) for z in
  ↪  f[1:]),key=repr))
assert canon(prep)==canon(q33)
assert canon(gpat)==canon(wrap(rawg)) and canon(hpat)==canon(wrap(rawh))
assert len(patterns)==192 and len(prefix)==65
assert all(size(t)<=i+4 for i,t in enumerate(prefix,1))
assert all(not emb(s,t) for i,s in enumerate(prefix) for t in prefix[i+1:])
assert all(not match(t,wrap(f)) for t in prefix for f in patterns)
assert all(not match(t,prepat) for t in prefix[:64])
assert all(not match(seed(f),g) for i,f in enumerate(patterns)
           for g in patterns[:i])
assert all(not match(seed(prep),f) for f in patterns)
profile=[check_skeleton(f) for f in patterns]
expected=[(4,0,15),(3,1,15),(3,0,30),(2,2,11),(2,1,28),
          (2,0,38),(1,3,3),(1,2,12),(1,1,21),(1,0,19)]
start=0
for d,k,num in expected:
 assert all(x[:2]==(d,k) for x in profile[start:start+num]);start+=num
assert start==192 and sum(z[2] for z in profile)==363
def fill_one(f,t):
 return t if f[0]=='hole' else tree(*(fill_one(z,t) for z in f[1:]))
def Q(t):return whole(fill_one(prep,t))
qs=Q(S2)
assert all(not match(t,hpat) and not match(t,gpat)
           for t in prefix[:64]+(qs,))
def BB(t):return tree(t,U(A))
H=whole(tree(tree(BB(p),BB(p)),tree(BB(p),BB(S2))))
GG=whole(tree(tree(BB(p),BB(p)),tree(BB(p),BB(p))))
assert not match(H,gpat)
assert all(not match(t,wrap(f)) for t in (H,GG)
           for f in patterns)
assert (size(H),size(GG),size(Q(p))-1)==(72,70,64)
assert prefix[-1]==Q(tree(p,tree(p,p)))
assert (size(D0),size(whole(p))-1)==(23,39)

# Earlier intervals, including the high-degree block.
```

```
assert all(not match(t,positive(aa)) for t in prefix[12:])
atoms=union(N(N(),N(),N(('chain',0),('chain',1))),
            N(N(),N(),N(N(('chain',0),('chain',1)))),
            fixed(M(U(S2,2))),fixed(h))
@lru_cache(None)
def wide(s):
 return match(s,atoms) or len(s)<=2 and all(map(wide,s))
assert all(not wide(t) for t in prefix[:12])
nu0=N(fixed(D0),N(('Z',),('Z',)))
nu1=N(fixed(D0),N(N(('Z',),('Z',))))
ep=N(fixed(D0),N(N(),N(fixed(A2),N(('T1',),fixed(A2)))))
assert all(not match(t,f) for t in prefix for f in (nu0,nu1,ep))
assert all(not match(whole(p),f) for f in (nu0,nu1,ep))
assert not match(whole(p),positive(aa))

def poly_pair(a):
 b={}
 for i,x in a.items():
  for j,y in a.items():b[i+j]=b.get(i+j,0)+x*y
  b[2*i]=b.get(2*i,0)+x
 assert all(x%2==0 for x in b.values())
 return {i:x//2 for i,x in b.items()}
poly={0:138,1:240,2:200,3:75,4:15}
for add in (1,1,0):poly=poly_pair(poly);poly[0]+=add
assert max(poly)==32 and 10000*sum(poly.values())<2**128
def L21(q):
 a,r=divmod(q,2)
 return (r+3)*(2**(a+1)-1)-2*(a+1)
t=0;ends=[]
for j in range(4):t+=L21(j+t);ends.append(t)
assert ends==[1,6,89,3*2**47-8]
assert 65+t-66>=64*(65536+2)
assert 6**100>2**16*5**100
assert 2**1024>=(2**130*(1024+2))**2
print('PASS: 65 entrance trees, 192 families, 363 skeleton maps;')
print('all reverse-family and entrance grammar decisions;')
print('high-template obligations and direct bootstrap arithmetic.')
```